

CSC113 PROJECT
PHASE2

ICE CREAM SHOP SYSTEM

SECOND SEMESTER - 1446
SUPERVISED BY : Dr.tasneem Alowaisheq
Section.No : 27891

NAME	STUDENT ID	WORK DIVISION
NOURA ALMUAYLI	445201323	Everything was group work
NOURA ALTUWAIM	445202179	
ALANOUD ALSANAD	445202240	

INTRODUCTION

PROJECT IDEA:

Ice Cream Shop System

PROJECT INTRODUCTION :

The Ice Cream Shop System is a Java-based application that simulates the core functions of a real ice cream store. The system allows users to create and manage customer carts, add a variety of ice cream types including regular options like vanilla and chocolate, as well as sugar-free alternatives , calculate total prices based on quantity and scoops, and perform checkout operations. It supports multiple carts and keeps track of each cart's individual items and total cost.

IMPROVEMENTS BETWEEN PHASES

1.Exception Handling:

- Custom exceptions (InvalidCartException) for invalid operations.
- Built-in exceptions (NumberFormatException, IOException) for input validation and file operations.

2.Data Persistence:

- Automatically saves shop summary (Shop_Summary.txt) and detailed cart contents (Cart_X_Details.txt) in human-readable format upon exit.

•

3.User-Friendly GUI:

- Main Menu: Welcome screen with options to manage carts or place orders.
- Cart Management Frame: View, checkout, or empty carts.
- Order Frame: Add, remove, or search ice cream items.

4.Improved Data Structures:

- In Phase 2, we replaced arrays with a Node-based linked list for storing ice cream items, enhancing flexibility and scalability.
- Each Cart now maintains a headIceCream (starting node) of the linked list.
- Ice cream items are serialized (saved to files) in the linked list structure.

5.Simplified Cart Operations

- Removed redundant features like cart deletion, focusing instead on checkout and emptying carts.
- Added a "View All" option to inspect all carts at once.

DESCRIPTION

1. IceCream (Abstract Class)

Purpose: Base class for all ice cream types

Attributes:

- iceCreamId: String - Unique identifier for the ice cream
- flavor: String - Flavor of the ice cream (e.g., Vanilla, Chocolate)
- quantity: int - Number of items ordered

Methods:

- calculatePrice(): abstract double - Calculates the price (implemented by subclasses)
- toString(): String - Returns ice cream details
- Getters/Setters for all attributes

2. RegularIceCream (Abstract)

Extends: IceCream

Attributes:

- scoops: int - Number of scoops

Methods:

- getScoops()/setScoops(): Manage scoops
- toString(): Appends scoops and type to superclass output

3. SpecialIceCream (Abstract)

Extends: IceCream

Attributes:

- scoops: int - Number of scoops

Methods:

- Same as RegularIceCream but labels type as "Special"

4. Flavor Classes (Vanilla, Chocolate, SugarFreeVanilla, SugarFreeChocolate)

Extend: RegularIceCream or SpecialIceCream

Key Methods:

- calculatePrice(): Implements price logic:
 - Regular Vanilla: $(5 * \text{scoops}) * \text{quantity}$
 - Regular Chocolate: $(7 * \text{scoops}) * \text{quantity}$
 - Sugar-Free Vanilla: $(10 * \text{scoops}) * \text{quantity}$
 - Sugar-Free Chocolate: $(15 * \text{scoops}) * \text{quantity}$
- toString(): Includes flavor-specific pricing

5. Cart

Purpose: Manages a shopping cart with enhanced text-based persistence

Attributes:

- cartId: String - Unique cart identifier (1-5)
- headIceCream: Node - Start of linked list for ice cream items
- maxItems: int (constant=10) - Maximum capacity of ice creams per cart

Key Methods:

1.addIceCream(IceCream):

- Adds ice cream to cart
- Throws InvalidCartException if cart is full (≥ 10 items) or duplicate ID exists
- Auto-sorts items by addition time (newest first)

DESCRIPTION

2.removeIceCream(String iceCreamId):

- Removes specific ice cream by ID
- Returns boolean success status

3.searchIceCream(String iceCreamId):

- Retrieves ice cream by ID
- Returns IceCream object or null

4.calculateTotal():

- Calculates sum of all items' prices
- Returns formatted double (2 decimal places)

5.saveIceCreamData():

- Persists cart to Cart_X_Details.txt with

6.loadIceCreamData():

- Restores cart from formatted text file
- Maintains item order and all attributes

6.Shop

Purpose: Manages all shopping carts with store-wide persistence

Attributes:

- shopName: String - Store identifier ("Sweet Treats")
- carts: Cart[] - Array of active carts (max 5)
- numCarts: int - Current number of active carts
- maxCarts: int (constant=5) - Maximum carts allowed

Key Methods:

1.addCart(Cart):

- Registers new cart to the system
- Auto-assigns sequential IDs (1-5)

2.emptyCart(String cartId):

- Clears all items from specified cart
- Returns boolean success status

3.searchCart(String cartId):

- Finds cart by ID (1-5)
- Returns Cart object or null

4.saveCartsData():

- Saves shop overview to Shop_Summary.txt

5.loadCartsData():

- Restores shop state from text files:
 - i. Reads Shop_Summary.txt for basic info
 - ii. Loads individual Cart_X_Details.txt for each cart

6.getInventoryReport():

- Generates formatted string of all carts' statuses
- Includes item counts and total values

DESCRIPTION

7. Node

Purpose: Linked list node for ice creams

Attributes:

- data: IceCream - Ice cream object
- next: Node - Reference to next node

Methods:

- Standard getters/setters

8. InvalidCartException

Purpose: Custom exception for cart errors

Key Cases:

- Invalid cart ID (not 1-5)
- Full cart (>10 items)
- Empty cart operations

9. GUI Classes (Frame1, CartsFrame, OrderFrame)

Shared Features:

- Frame1: Main menu with buttons for navigation
- CartsFrame:
 - Checkout/Empty carts
 - View all carts in JTextArea
- OrderFrame:
 - Add/Remove/Search ice creams
 - Dynamic JComboBox for types/flavors
- All frames:
 - Auto-save on exit
 - Input validation

10. TestIceCreamShop

Purpose: Entry point with console/GUI modes

Key Features:

- Initializes 5 default carts
- Implements all menus (manage carts, order ice cream)
- Handles file I/O on startup/exit

EXCEPTION HANDLING

1. Custom Exceptions (Checked)

InvalidCartException (Checked - Custom)

- Purpose: Handles invalid cart operations
- Triggered When:
 - Invalid cart ID (not 1-5)
 - Attempt to add to a full cart (>10 items)
 - Operations on empty carts
- Files:
 - Cart.java (e.g., addIceCream() throws when cart is full)
 - CartsFrame.java (checkout with invalid ID)
 - OrderFrame.java (adding to full cart)

2. Built-in Unchecked Exceptions

1.NumberFormatException (Unchecked - Built-in)

- Purpose: Invalid numeric input (e.g., text entered where numbers are required)
- Files:
 - CartsFrame.java (cart ID parsing)
 - OrderFrame.java (ice cream ID/quantity/scoops parsing)
 - TestIceCreamShop.java (console input validation)

2.InputMismatchException (Unchecked - Built-in)

- Purpose: Incorrect input type (e.g., string instead of number)
- Files:
 - TestIceCreamShop.java (console menu inputs)
 - OrderFrame.java (quantity/scoops input)

3. Built-in Checked Exceptions (I/O)

1.IOException

- Purpose: General file operation failures
- Used in:
 - Cart.java (saveIceCreamData(), loadIceCreamData())
 - Shop.java (saveCartsData(), loadCartsData())

2.FileNotFoundException

- Purpose: Missing data files
- Used in:
 - Cart.java (loadIceCreamData() - if IceCream_X.txt missing)
 - Shop.java (loadCartsData() - if Shop_Summary.txt missing)

3.ClassNotFoundException

- Purpose: Corrupted/incompatible data
- Used in:
 - Cart.java (loadIceCreamData())
 - Shop.java (loadCartsData())

FILE MANAGEMENT

1. File Structure

File Type :

Shop Summary

Cart Details

Filename:

Shop_Summary.txt

Cart_X_Details.txt

FormatContents:

Total carts, last update time, cart summaries

All ice cream items in cart (X = cart ID)

2. Key Operations

1. Saving Data

- Triggered when:
 - Program exits
 - Cart checkout occurs

2. Loading Data

- Triggered at program startup
- Process:
 - a. Reads Shop_Summary.txt first
 - b. Loads individual Cart_X_Details.txt files

3. File Format Highlights

- Structured with clear sections:

=== Cart Information ===

Cart ID: 1

Total Items: 3

=== Ice Cream Items ===

Item #1

Flavor: Chocolate

Type: Regular

Price: \$7.00

4. Error Handling

- Automatically creates new files if none exist
- Skips corrupted files during loading
- Shows error messages if saving fails

IMPLEMENTATION

1. Main Menu (Frame1)

Purpose: The starting point that lets users choose what they want to do.

What the user sees:

- Large title: "Welcome To The Sweet Treats Shop!"
- 3 main buttons:
 - a. Manage Carts - Opens cart management
 - b. Order Ice Cream - Opens ordering system
 - c. Exit - Saves data and closes program

What happens when clicking:

- Buttons open their respective screens
- Exit button saves all data before closing

2. Cart Management (CartsFrame)

Purpose: Lets users view and manage shopping carts.

What the user sees:

- Left Panel (Cart Operations):
 - Cart ID input field
 - "Checkout" and "Empty" buttons
- Right Panel (View Carts):
 - Text display area
 - "View All" button

What happens when clicking:

- Checkout: Shows cart contents and total price, then empties it
- Empty: Clears all items from selected cart
- View All: Displays all added carts with their items

3. Order Ice Cream (OrderFrame)

Purpose: Lets users add/remove ice creams from carts.

What the user sees:

- Left Panel (Order Form):
 - Cart ID field
 - Ice cream details (ID, type, flavor, quantity, scoops)
 - Add/Remove/Search buttons
- Right Panel (View Orders):
 - Large display area
 - "View Order" button

What happens when clicking:

- Add: Creates new ice cream item in selected cart
- Remove: Deletes ice cream by ID
- Search: Finds and shows specific ice cream
- View Order: Displays all items in selected cart

GUI COMPONENTS

- **Labels (JLabel)**

Used for displaying text like “Cart ID”, “Ice Cream ID”, “Quantity”, and guiding the user through the form.

- **Buttons (JButton)**

Includes buttons such as:

- Add Cart, Remove Cart, Search Cart, Checkout Cart
- Add Ice Cream, Remove Ice Cream, Search Ice Cream, Back

- **Text Fields (JTextField)**

Used for user input: cart ID, ice cream ID, quantity, and number of scoops.

- **Combo Box (JComboBox)**

Used for selecting ice cream type (Regular / Special) and flavor (Vanilla, Chocolate, etc.).

- **Text Area (JTextArea)**

Displays full cart content, ice cream order summaries, and search results.

- **Scroll Pane (JScrollPane)**

Enables scrolling for text areas containing long outputs.

- **Panel (JPanel)**

Organizes all components inside each frame.

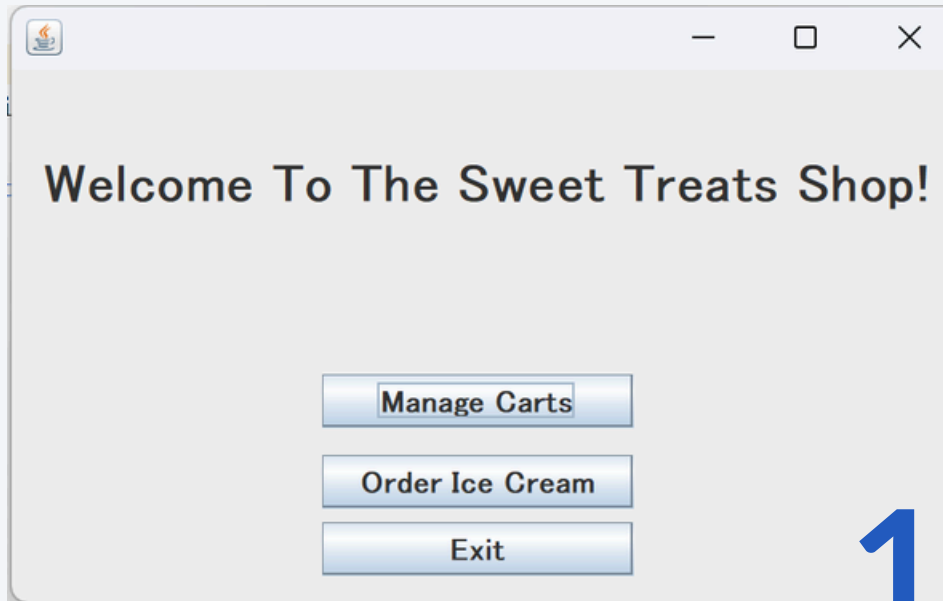
- **Dialog Boxes (JOptionPane)**

Used to show error messages, confirmations, and success alerts.

GUI SCREENSHOTS

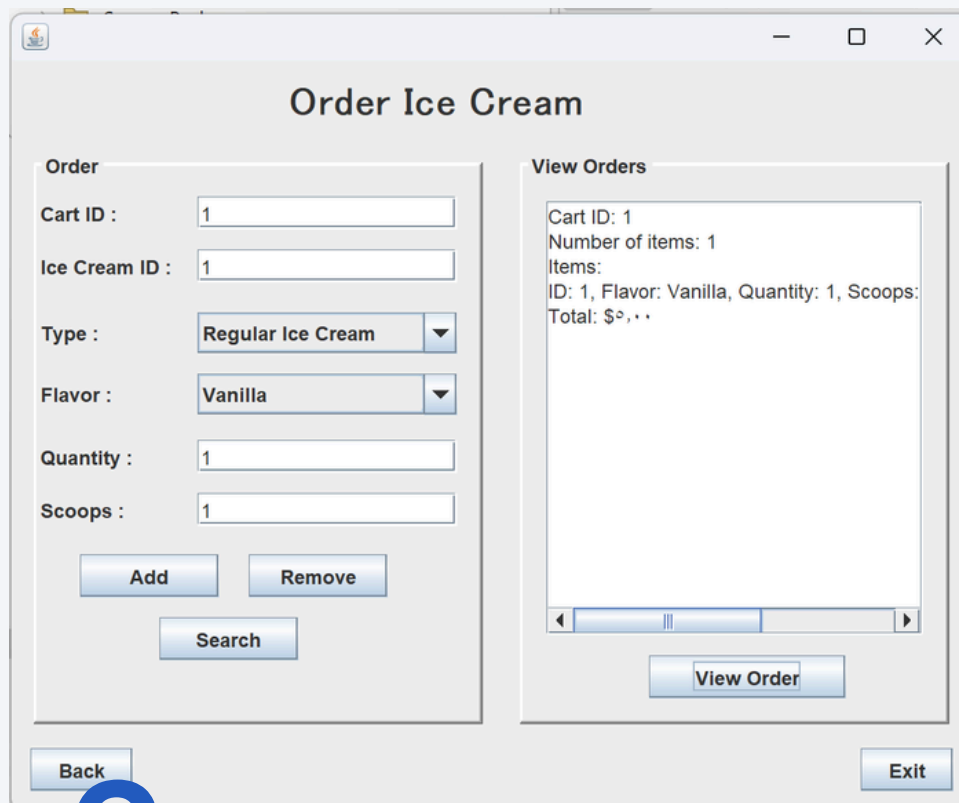
in our GUI we have 3 frames:

1. Main Welcoming frame
2. Ordering an Ice Cream frame
3. Managing cart frame



Main frame

1



2

Ordering an ice cream frame

Order Ice Cream

Order
Cart ID :
Ice Cream ID :
Type :
Flavor :
Quantity :
Scoops :

View Orders
Cart ID: 1
Number of items: 1
Items:
ID: 1, Flavor: Vanilla, Quantity: 1, Scoops:
Total: \$0.00

Message

i Ice cream added successfully!
ID: 1, Flavor: Vanilla, Quantity: 1, Scoops: 1, Type: Regular, Price: \$5.0

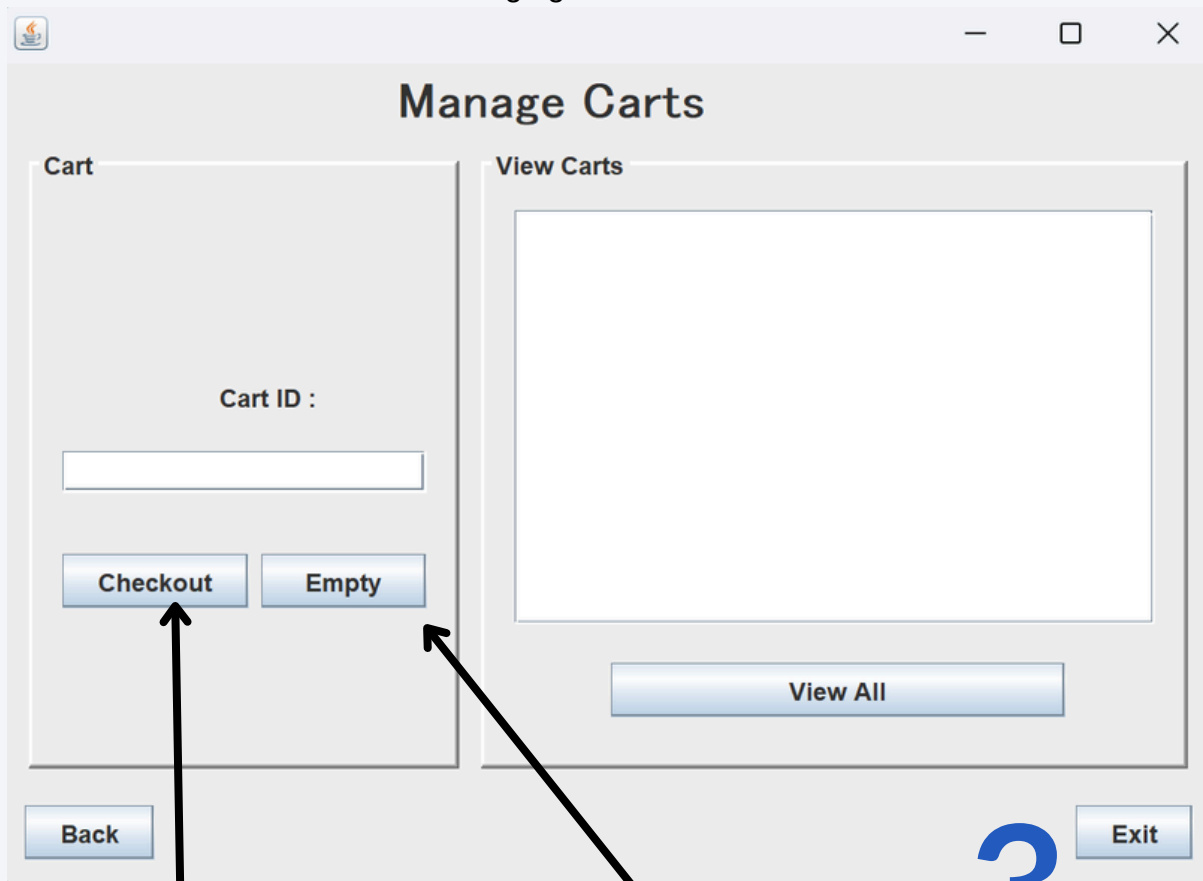
Message

i Ice cream removed successfully!

Message

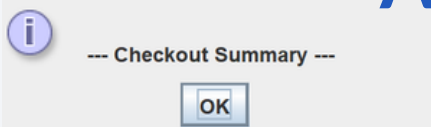
i Ice Cream Details:
ID: 1, Flavor: Vanilla, Quantity: 1, Scoops: 1, Type: Regular, Price: \$5.0

Managing cart frame



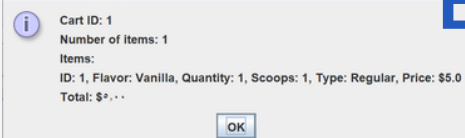
The 'Managing cart frame' window is divided into two main sections: 'Cart' and 'View Carts'. The 'Cart' section contains a 'Cart ID :' label above a text input field, and two buttons labeled 'Checkout' and 'Empty'. The 'View Carts' section contains a large empty rectangular area and a 'View All' button at the bottom. At the bottom of the window are three buttons: 'Back', 'Exit', and a large blue number '3'. Arrows point from the 'Checkout' button to a message box labeled 'A' and from the 'Empty' button to a message box labeled 'B'.

Message



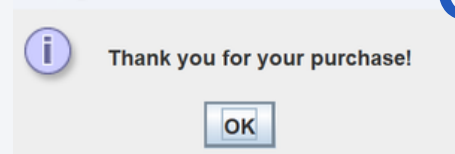
A

Message



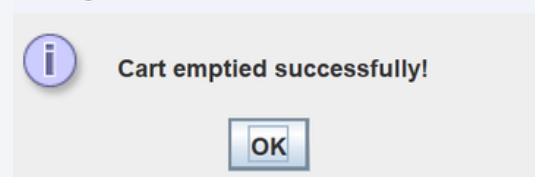
B

Message



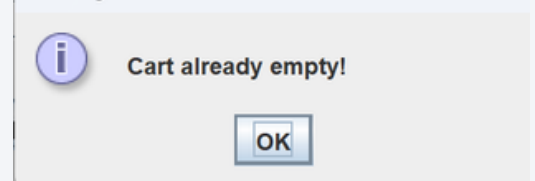
C

Message

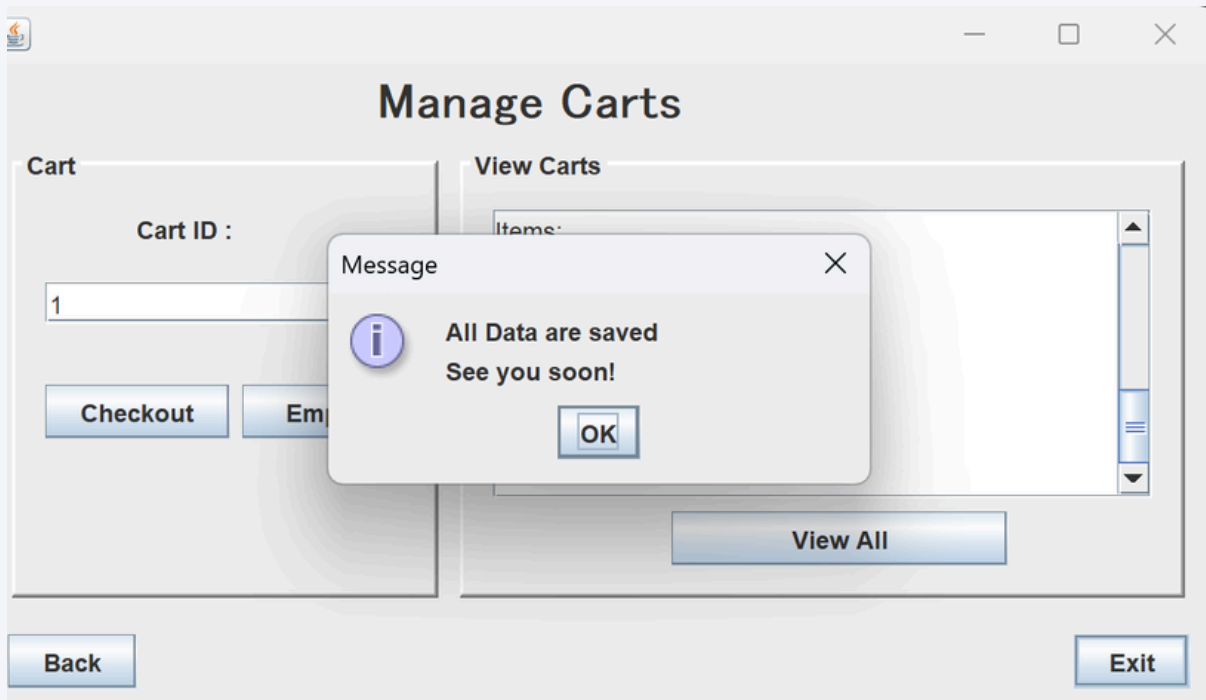


OR

Message



Exiting message



Ex.Saved Data

```
=== Cart Information ===
Cart ID: 1
Total Items: 1
0,00$:Total Price
=====
=== Ice Cream Items ===

Item #1
ID: 1
Flavor: Vanilla
Quantity: 1
Type: Regular
Scoops: 1
Price: $5.0
```

```
=== Sweet Treats Shop Summary ===
Total Carts: 5
Last Updated: Sat May 03 21:53:17 GMT+03:00 2025
=====
Cart ID: 1
Items Count: 1
0,00$:Total Value
Cart ID: 2
Items Count: 0
0,00$:Total Value
Cart ID: 3
Items Count: 0
0,00$:Total Value
Cart ID: 4
Items Count: 0
0,00$:Total Value
Cart ID: 5
Items Count: 0
0,00$:Total Value
```